RestTemplate - Synchronous Communication

<u>In this topic today, will cover:</u>

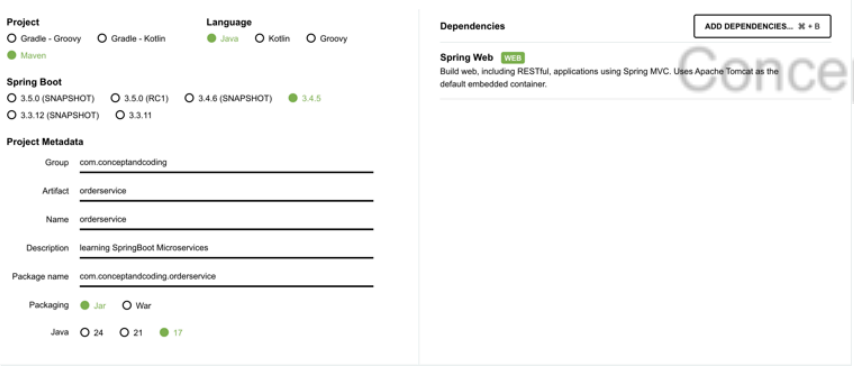
1st : Set up of 2 microservice i.e. "OrderService" and "ProductService" running on different port numbers.

2nd : How two microservices can communicate with each other

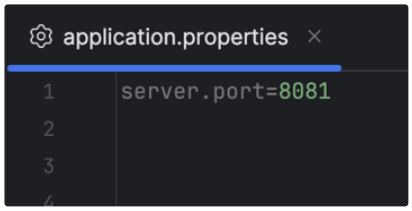
<u>Let's start:</u>

1st : Set up of 2 microservice

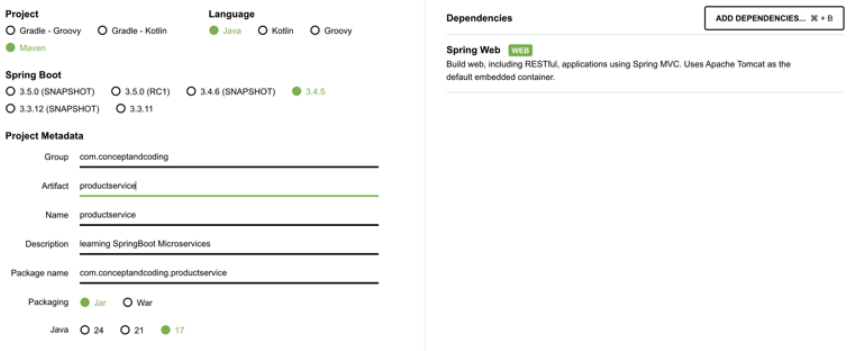
OrderService: Go to Spring Initializer (start.spring.io)

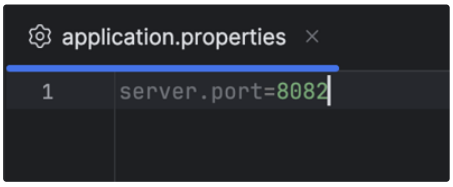


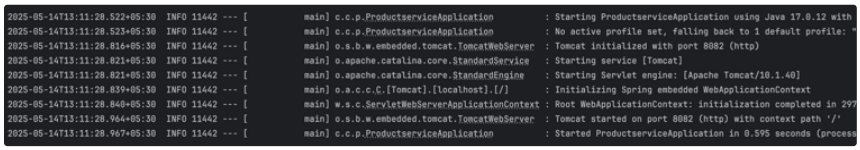
Concept && Coding



Similarly, set up ProductService







Concept && Coding

2nd : Communication between 2 services

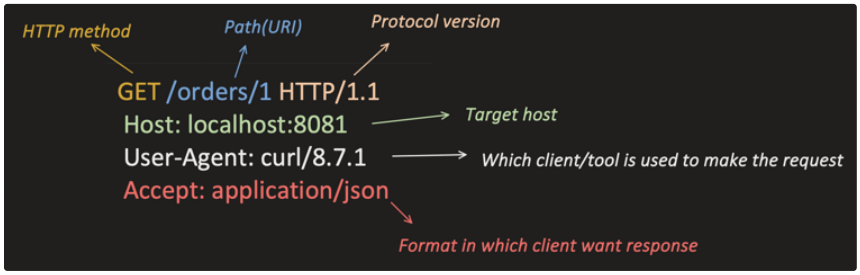
Synchronous Communication :

- Client wait for the response from the Server before continuing.
- Blocking in nature, means thread waits till response is not received.

Synchronous Communication Types in SpringBoot:

- RestTemplate
- RestClient
- FeignClient

<u>Sample HTTP GET Request call:</u>



Sample HTTP POST Request call:

```
POST /products HTTP/1.1
Host: localhost:8081
User-Agent: curl/8.7.1
Accept: application/json
Content-Type: application/json        → Tell server that, request body is JSON
Content-Length: 65                    → Tells the number of bytes in the request body
{
 "name": "Ice-Cream",                 → Actual data, a new product object
 "price": 200
}
```

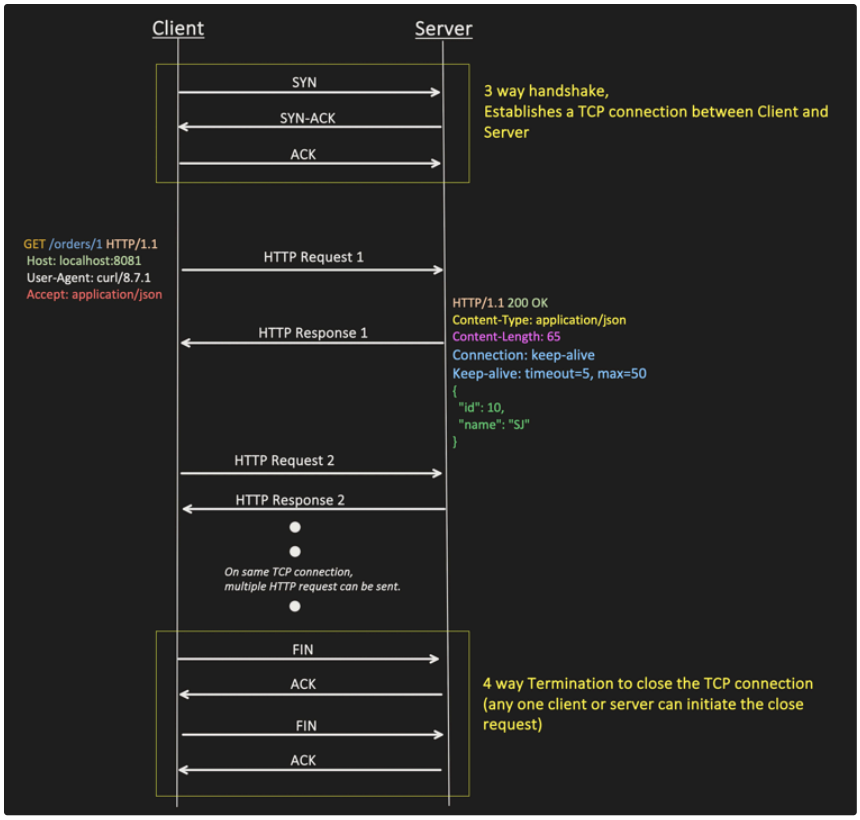
Sample HTTP GET Response call, with keep alive:

```
                                      2xx : request is successful
                                      4xx: client sent invalid request
                                      5xx: Server error
HTTP/1.1 200 OK
Date: Fri, 16 May 2025 10:00:00 GMT
Content-Type: application/json        ← Tells client to expect JSON response
Content-Length: 65                    → No of bytes in response body
Connection: keep-alive
Keep-alive: timeout=5, max=50
{
 "id": 10,                            → Response body
 "name": "SJ"
}
```

Concept && Coding

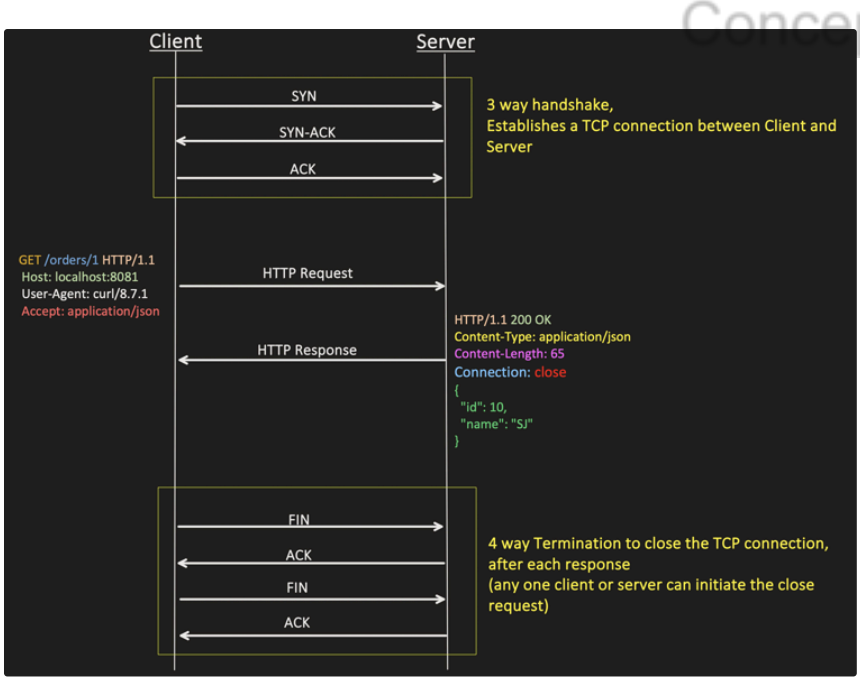
- By default, connection is set to keep-alive in HTTP/1.1
- In HTTP/1.0 by-default connection is set to close.
- Keep-alive:
  - timeout=5, tells close the TCP connection if its idle for 5 seconds
  - Max=50, tells the maximum number of requests can be send over same TCP connection.
- When Connection: close is set, it tells after every response from the server, TCP connection is closed, its not reused.

Flow, when keep-alive is set:



Flow, when connection: close is set:

Concept && Coding



Let's first see, without using above SpringBoot communication types, what it takes to invoke the REST endpoint just using plain JAVA.

OrderService:

```
@RestController
```

```java
@RequestMapping("/orders")
public class OrderController {

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {

        HttpURLConnection httpURLConnection = null;
        try {
            String url = "http://localhost:8082/products/" + id;

            URL obj = new URL(url);
           /*
            Creates an Object of HttpURLConnection,
            consider it like an envelop or request,
            in which we specify all the details like URL,
            Request Method, timeouts etc.
            */
            httpURLConnection = (HttpURLConnection) obj.openConnection();

            // Setting http request method and header
            httpURLConnection.setRequestMethod("GET");
            httpURLConnection.setRequestProperty("Accept", "application/json");

            // max time to establish TCP connection (ms)
            httpURLConnection.setConnectTimeout(100);
            // max time to wait for server response (ms)
            httpURLConnection.setReadTimeout(500);

            /*
               Here it opens up a TCP connection and send the HTTP request,
               also reads the response.
              TCP connection is created, when we make the
              first Input/Output request on HttpURLConnection such as:
                - getInputStream()
                - getResponseCode()
                - connect() etc.…
            HttpClient object is a wrapper around TCP connection,
            so before creating new HttpClient object,
            if first checks with "KeepAliveCache" class,
            if there is already an object present,
            if not it creates one object and also puts into the cache.
            key -> host:port
            value -> httpClient object
            */
            BufferedReader in =
                    new BufferedReader(new InputStreamReader(httpURLConnection.getInputStream()));

            StringBuilder response = new StringBuilder();
            String responseLine;

            while ((responseLine = in.readLine()) != null) {
                response.append(responseLine);
            }

            in.close();
            System.out.println("Response: " + response.toString());

        } catch (Exception e) {
            // exception handling here
        } finally {
            if (httpURLConnection != null) {
              /*
                  If Response is fully read properly,
                  the TCP Connection i.e. HttpClient is returned
                  back to KeepAlive cache else
                  TCP connection get closed. Framework code below:
                  public void disconnect() {
```

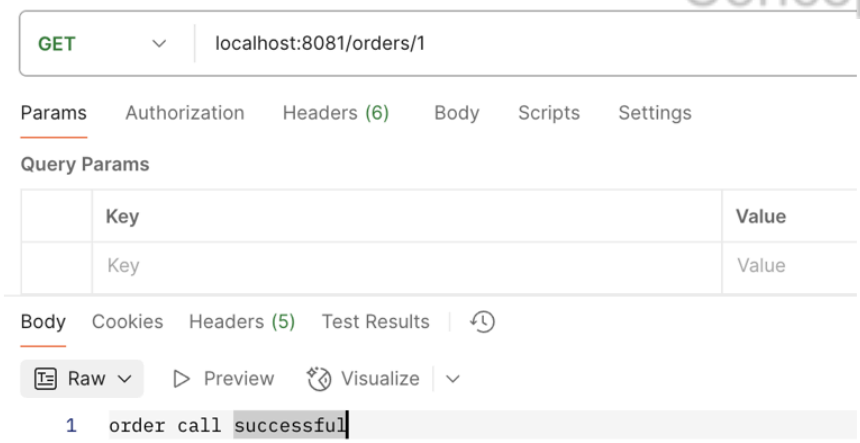
Concept && Coding

```
                        if (input Stream Not Fully Read) {
                            httpClient.closeServer(); // Close TCP socket
                        } else {
                          httpClient.finished();    // Return to KeepAlive Cache
                        }
                    }
                */
                  httpURLConnection.disconnect();
              }
          }

          return ResponseEntity.ok("order call successful");
      }
}

```

ProductService:

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public String getProduct(@PathVariable String id) {
        return "Product fetched with id: " + id;
    }
}

```

Concept && Coding



Couple of Disadvantage of above approach is:

- Too much Boilerplate code:
  - Open connection

    - Setting headers
    - Reading response
    - Closing streams and connections.
- Response should be handled manually.
    - No automatic mapping to some Objects.
- Limited support for Advance features like
    - Connection pooling
    - Interceptors etc.

RestTemplate:

- Abstract low level code like creating HttpURLConnection object etc.
- Traditional/Legacy way to call REST APIs in Spring application.

OrderService:

```java
@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

// Or, use below, if we want to set timeouts too

@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {

        SimpleClientHttpRequestFactory factory =
                new SimpleClientHttpRequestFactory();

        // timeouts in milliseconds
        factory.setConnectTimeout(1000);
        factory.setReadTimeout(5000);

        return new RestTemplate();
    }
}


```

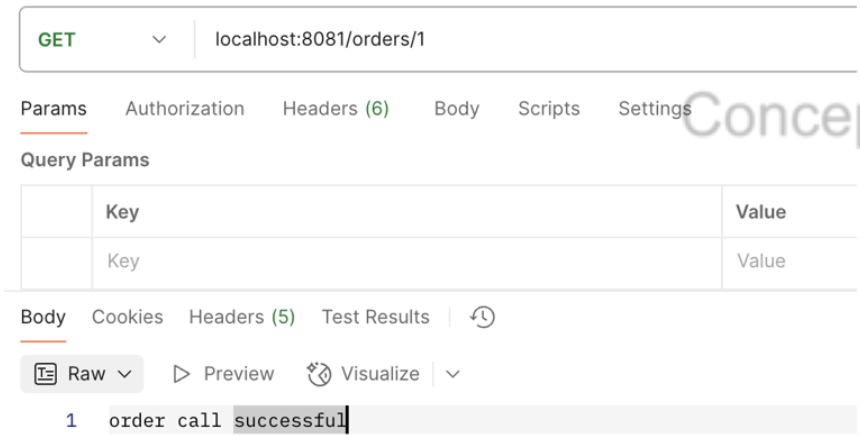
Concept && Coding

```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    RestTemplate restTemplate;

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {

        String response = restTemplate.getForObject(
                "http://localhost:8082/products/" + id,
```

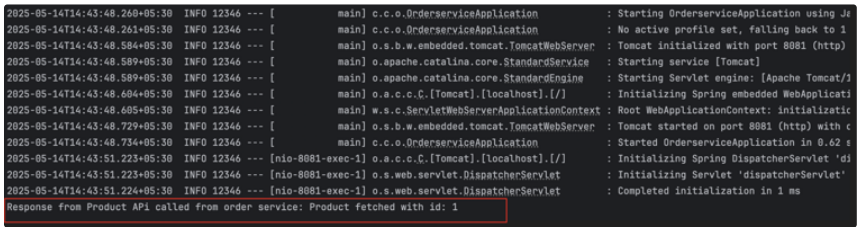
```
                String.class
        );

        System.out.println(
                "Response from Product API called from order service: " + response
        );

        return ResponseEntity.ok("order call successful");
    }
}

```

ProductService

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public String getProduct(@PathVariable String id) {
        return "Product fetched with id: " + id;
    }
}

```



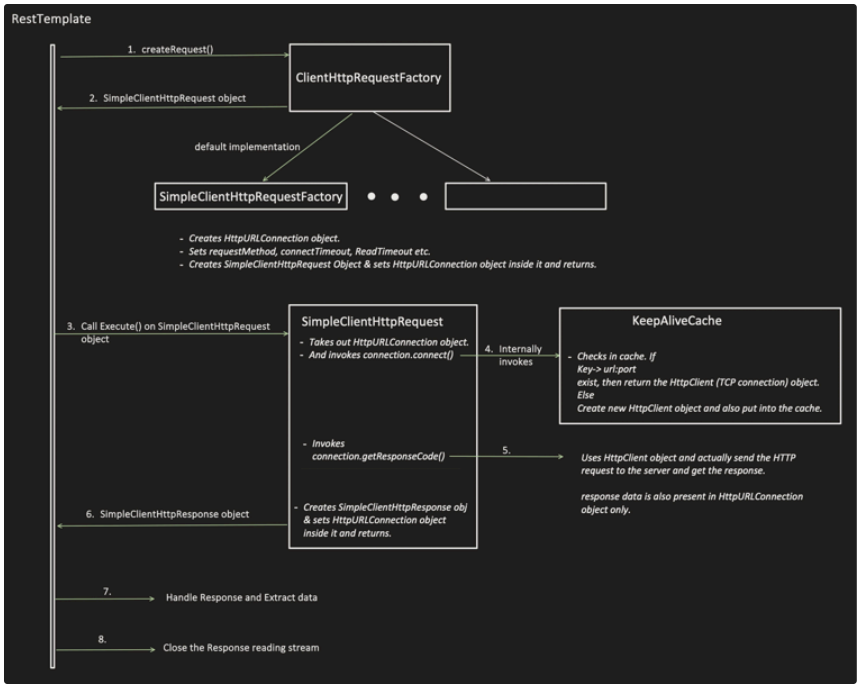
Concept && Coding



So, what exactly happened, RestTemplate works internally:

When below method *"getForObject"* invoked:

String response = restTemplate.getForObject("http://localhost:8082/products/"+id, String.class);



Notice one thing that:

- RestTemplate do not close the TCP connection explicitly.
- Once Response stream is closed, TCP connection (i.e. HttpClient object, is ready to be re-used till is not expired based on idle Timeout or max Connection configuration)

Concept && Coding

Lets see, some of the other methods which are available in RestTemplate

Method Name	Description
GET	
getForObject(String url, Class<T> responseType)	Returns the response body as an Object. String url = "http://localhost:8080/api/products/1"; Product product = restTemplate.getForObject(url, Product.class);
getForEntity(String url, Class<T> responseType)	Returns full ResponseEntity with status and header String url = "http://localhost:8080/api/products/1"; ResponseEntity<Product> response = restTemplate.getForEntity(url, Product.class); HttpStatus status = response.getStatusCode(); Product product = response.getBody();

POST	
postForObject(String url, Object request, Class<T> responseType)	Sends POST and get just the response body. String url = "http://localhost:8080/api/products"; Product newProduct = new Product("Ice-cream", 100); Product createdProduct = restTemplate.postForObject(url, newProduct, Product.class);
postForEntity(String url, Object request, Class<T> responseType)	Sends POST and get just the full ResponseEntity object. String url = "http://localhost:8080/api/products"; Product newProduct = new Product("Ice-cream", 100); ResponseEntity<Product> response = restTemplate.postForEntity(url, newProduct, Product.class); Product createdProduct = response.getBody(); HttpStatus status = response.getStatusCode();
PUT	
put(String url, Object request)	Sends PUT and no response body is expected. String url = "http://localhost:8080/api/products/1"; Product updatedProduct = new Product("Ice-cream", 150); restTemplate.put(url, updatedProduct);
DELETE	
delete(String url)	Sends DELETE request and no response body is expected. String url = "http://localhost:8080/api/products/1"; restTemplate.delete(url);
GENERAL PURPOSE	
exchange(String url, HttpMethod method, HttpEntity<?> requestEntity, Class<T> responseType)	When we want to customize : • HTTP method (GET, PUT, POST etc.) • HTTP header and body (HttpEntity) • But want Spring automatic Conversion

Concept && Coding

	String url = "http://localhost:8080/api/products/";
	
	//customizing header
	HttpHeaders headers = new HttpHeaders();
	headers.setContentType(MediaType.APPLICATION_JSON)headers.set("Authorization", "Bearer my-token");
	
	//preparing http request body
	Product product = new Product();
	product.setName("Ice-cream");
	product.setPrice(100);
	//setting both header and body in the HttpEntity
	HttpEntity<Product > requestEntity = new HttpEntity<> (product, headers);
	
	ResponseEntity<Product> response = restTemplate.exchange(
	  url,
	  HttpMethod.POST,
	  requestEntity,
	  Product.class
	);
	
	Product product = response.getBody();
	HttpStatus status = response.getStatusCode();

execute(String url, HttpMethod method, RequestCallback requestCallback, ResponseExtractor<T> responseExtractor)	When we want full control like in plain java we use HttpURLConnection object. Header, body, Request, Response, serialization etc. need to be handled manually. RequestCallback interface, gives us full control over the request. We can set header, write body etc. @FunctionalInterface public interface RequestCallback {

Concept && Coding

```
    void doWithRequest(ClientHttpRequest request) throws IOException;
}
```

Similarly, ResponseExtractor, gives us full control over, how the response is read and converted to desired object.

```
@FunctionalInterface
public interface ResponseExtractor<T> {
    T extractData(ClientHttpResponse response) throws IOException;
}
```

```
RestTemplate restTemplate = new RestTemplate();

String url = "http://localhost:8080/api/products";

//setting both header and body
RequestCallback requestCallback =  request -> {

request.getHeaders().setContentType(MediaType.APPLICATION_JSON);
    Product product = new Product("Ice-cream", 100);
    ObjectMapper mapper = new ObjectMapper();
    byte[] body = mapper.writeValueAsBytes(product);
    StreamUtils.copy(body, request.getBody());
};

//parsing the response
ResponseExtractor<String> responseExtractor = response -> {
    return StreamUtils.copyToString(response.getBody(), StandardCharsets.UTF_8);
};
```

Concept && Coding

	String response = restTemplate.execute(
	  url,
	  HttpMethod.POST,
	  requestCallback,
	  responseExtractor
	);
	
	System.out.println("response is: " + response);

Limitation of RestTemplate:

- In RestTemplate, there are already so many overloaded methods, so its hard to remember and maintain.(Above we have just covered few)
- RestTemplate was build before concepts like Retry, circuit breaker etc.. So adding support means more overloaded methods and not user friendly.
- RestTemplate is in Maintenance mode - means no new feature, only bug fixes.

Concept && Coding

That's where latest RestClient comes into the picture:

- Introduction of Fluent, builder-style API (more readable and user friendly way of configuring and invoking the endpoint)
- RestClient supports easy integration with interceptors, filters etc.